

Stage 1 MVP Specification

3 단계 하네스 중 Planning Harness MVP 를 실제 구현하기 위한 문서화된 명세

CLI-first / Artifact-first / Gate-driven

문서 목적

기존 3 단계 검토 보고서에서 도출된 방향을 Stage 1 범위로 재구성하여, 곧바로 구현 가능한 수준의 제품 범위, 아키텍처, CLI 계약, artifact schema, planning gate, acceptance test, 저장소 구조를 확정한다.

항목	내용
문서 버전	v1.0
문서 상태	Proposed / Implementation-ready
작성일	2026-04-11
대상 범위	Stage 1 Planning Harness MVP
기본 인터페이스	Python CLI
핵심 산출물	seed.yaml / plan.md / task-graph.json / planning-validation.json / open-questions.md / run-ledger.json
비범위	코드 실행, worktree dispatch, verifier, self-improvement loop, dashboard

1. 실행 요약

한 줄 결론

Stage 1 MVP 는 ‘질문을 잘하는 인터뷰 도구’가 아니라, 애매한 요청을 검토 가능한 planning bundle 로 바꾸고, 계획이 충분히 준비되지 않았을 때 실행을 막아 세우는 planning runtime 이어야 한다.

이번 명세는 제공된 기존 소스와 추가 검토 소스를 모두 다시 대조한 뒤, Stage 1 에 지금 바로 필요한 요소만 채택하고 나머지는 의도적으로 제외하는 방식으로 구성했다. 핵심은 다음 다섯 가지다.

- 명세 우선: 인터뷰와 seed spec 동결이 plan 생성보다 먼저이며, seed 가 없다면 plan 을 만들지 않는다.
- 문서 우선: planning 규칙, threshold, artifact schema 는 코드 안에 묻지 않고 policy 와 schema 파일로 외부화한다.
- 게이트 우선: planning readiness 는 ‘대충 충분해 보임’이 아니라 deterministic validator 가 판정한다.
- 작은 태스크 우선: task graph 는 DAG 여야 하며, 모든 태스크는 S 또는 M 크기로 제한한다.
- 확장 대비: Stage 2/3 기능은 구현하지 않지만, future handoff 를 위해 verify_hints, files_likely_touched, run ledger 필드를 남긴다.

MVP 의 정의는 ‘요청 → 인터뷰 → seed 동결 → plan 생성 → task graph 생성 → planning gate 판정 → bundle export’의 페루프가 안정적으로 동작하는 것이다. 코드 변경이나 에이전트 병렬 실행은 Stage 2 이후로 미룬다.

2. 소스 재검토 결과와 채택/유예 결정

아래 표는 이번 Stage 1 명세에 실제로 반영한 판단을 요약한다. 실행·개선 단계 소스도 모두 검토했지만, MVP 에는 필요한 인터페이스만 남기고 구현은 유예한다.

소스	결정	Stage 1 적용 방식
SlopCodeBench	채택	pass-rate 만으로 충분하지 않다는 경고를 받아들여, planning bundle 안에 scope/assumption/risk/acceptance 를 구조적으로 강제한다.
Paperclip	부분 채택	org chart/heartbeat/budget 은 Stage 2~3 로 유예. 다만 policy 파일과 run ledger 형태는 향후 control plane 확장을 고려해 설계한다.
jangpm blueprint	채택	설계 문서 우선, 문서 검증 스크립트, 섹션 누락 시 실패 처리 패턴을 Stage 1 validator 설계에 반영한다.
jangpm deep-dive	채택	다단계 probing interview 와 기존 spec 업데이트 우선 원칙을 반영한다.
Agent Skills	채택	spec before code, small atomic tasks, verifiable exit criteria 를 task

		graph 와 acceptance criteria 설계에 반영한다.
SIEGE	강하게 채택	gated planning, dependency-aware decomposition, policy-driven runtime 사고를 거의 그대로 가져온다.
Ouroboros	강하게 채택	specification-first workflow, immutable seed freeze, clarity 문제를 입력 단계에서 해결해야 한다는 점을 반영한다.
Superpowers	부분 채택	설계 승인 후 plan 작성, 사람에게 읽히는 설계 단위, 태스크 크기 제한을 반영한다. subagent 실행은 유예한다.
Autoresearch / autoresearch-skill	유예	Stage 3 의 mutation/promotion 로직으로 보관한다. 대신 Stage 1 부터 versioned artifact 와 changelog 친화 구조를 남긴다.
Natural-Language Agent Harnesses	강하게 채택	harness logic 을 자연어 artifact 와 explicit contract 로 외부화하는 접근을 stage policy 와 schema 파일 설계에 반영한다.
OpenAI Structured Outputs	채택	seed/task-graph/planning-validation 생성은 schema-constrained output 을 기본값으로 둔다.
OpenAI eval docs / best practices	채택	평가 없는 복잡도 증가는 피하고, multi-agent 는 eval 증거가 생긴 뒤 도입한다는 원칙을 채택한다.
LangSmith trajectory evals	부분 채택	Stage 3 주제이지만, Stage 1 부터 run-ledger 와 step trace 를 남기도록 설계한다.
Pydantic / JSON Schema / Typer	채택	Python CLI, 모델 기반 schema 생성, Draft 2020-12 계약을 MVP 기본 스택으로 권장한다.

이번 명세에서 의도적으로 하지 않는 것

온톨로지 기반 고급 ambiguity score, formal assurance, worktree execution, dashboard, multi-agent fan-out, prompt mutation loop 는 모두 Stage 1 성공 이후에만 붙인다. Stage 1 에서 가장 중요한 것은 planning bundle 의 품질과 gate 의 신뢰성이다.

3. 제품 정의

항목	내용
제품 한 문장 정의	사용자의 자연어 요청을 실행 가능한 planning bundle 로 변환하고, 준비가 부족하면 명시적으로 차단하는 Planning Harness
주 사용자	AI coding harness 를 운영하는 엔지니어, 테크 리드, 자동화 설계자
1 차 시나리오	새 기능/새 프로젝트 요청을 받아 seed spec 과 구현 계획을 만든 뒤, 인간 승인 후 Stage 2 실행기로 넘긴다.
2 차 시나리오	기존 spec bundle 을 다시 열어 update/resume 하고, 변경된 요구사항에 맞춰 seed/plan/task graph 를 재생성한다.
핵심 성공 상태	bundle 상태가 READY 가 되며 필수 artifact 와 planning-validation PASS/WARN 결과가 모두 존재한다.
핵심 실패 상태	필수 섹션 누락, untestable success criteria, task graph cycle, oversized task, 승인 누락

3.1 범위

- 입력: 자연어 request.md 또는 CLI 인자
- 인터뷰: clarity rubric 기반 다라운드 질의
- 산출물: seed.yaml / plan.md / task-graph.json / planning-validation.json / open-questions.md / run-ledger.json
- 검증: deterministic schema validation, DAG validation, gate rule evaluation
- 운영: signoff 이벤트 기록, bundle 버전 관리, resume/update 지원

3.2 비범위

- 실제 코드 수정, 테스트 실행, lint/build, worktree/container orchestration
- 여러 worker agent 를 병렬 fan-out 하는 시스템
- 자동 merge, production 배포, 비용 추정 엔진
- skill/prompt self-improvement loop 와 promote/discard 실험

3.3 완료 정의 (Definition of Done)

1. bundle 디렉터리에 필수 artifact 7 종 이상이 생성되어야 한다.
2. seed 와 JSON artifact 는 schema validation 을 통과해야 한다.
3. task-graph.json 은 DAG 여야 하며 모든 태스크가 S/M 크기여야 한다.
4. planning-validation.json 이 PASS 또는 WARN 이어야 하며, FAIL 이면 READY 로 승격할 수 없다.
5. seed freeze 와 plan signoff 이벤트가 run-ledger.json 에 기록되어야 한다.

4. 시스템 개요와 상태 전이

Stage 1 MVP 의 기본 제어 루프는 아래와 같다.

```

REQUEST_IN
-> EXISTING_BUNDLE_DISCOVERY
-> INTERVIEW
-> SEED_DRAFT
-> SEED_FROZEN
-> PLAN_DRAFT
-> TASK_GRAPH_DRAFT
-> PLANNING_VALIDATION
-> WAITING_SIGNOFF
-> BUNDLE_READY | BUNDLE_BLOCKED

```

핵심은 상태를 건너뛰지 않는 것이다. 특히 SEED_FROZEN 이전에 PLAN_DRAFT 로 진입할 수 없고, PLANNING_VALIDATION FAIL 상태에서는 BUNDLE_READY 로 승격할 수 없다.

상태	목적	출구 조건
REQUEST_IN	요청을 수신하고 run_id 를 발급	입력 파일 존재, 작업 디렉터리 확인
EXISTING_BUNDLE_DISCOVERY	관련 bundle 탐색	관련 bundle 발견 시 update/new 선택
INTERVIEW	부족한 정보만 질의	clarity 기준 충족 또는 interview budget 종료
SEED_DRAFT	schema-valid seed 생성	필수 필드 존재, parse 성공
SEED_FROZEN	seed 동결 및 버전 기록	freeze 승인 또는 operator override
PLAN_DRAFT	사람이 검토 가능한 plan.md 생성	architecture, milestones, risks, acceptance 포함
TASK_GRAPH_DRAFT	DAG task graph 생성	모든 노드 id 유일, deps 유효
PLANNING_VALIDATION	10 조건 gate 평가	PASS/WARN/FAIL 산출
WAITING_SIGNOFF	human approval 대기	seed/plan signoff 기록
BUNDLE_READY	Stage 2 handoff 가능	필수 artifact + signoff 완료

5. 모듈 명세

모듈	책임	권장 파일
CLI	stage1 run/interview/seed/plan/validate/signoff/resume 명령 제공	src/harness/cli.py
Bundle Discovery	기존 planning bundle 탐색 및 update/new 판단	src/harness/storage/discovery.py
Interview Engine	clarity rubric 기반 질문 생성, transcript 저장	src/harness/interview/engine.py
Assumption Extractor	default assumption, open question, risk 후보 추출	src/harness/interview/extractor.py
Seed Writer	seed schema 에 맞는 artifact 생성	src/harness/planner/seed_writer.py
Plan Generator	사람이 읽을 수 있는 plan.md 생성	src/harness/planner/

		plan_generator.py
Task Graph Builder	dependency-aware DAG 생성	src/harness/graph/builder.py
Planning Validator	schema, DAG, task size, gate rule 평가	src/harness/validation/planning_gate.py
Artifact Store	버전 폴더, manifest, run ledger 저장	src/harness/storage/bundle_store.py
Policy Loader	planning-policy.yaml, thresholds 로드	src/harness/policies/loader.py

5.1 모듈 인터페이스 권장 형태

```
class InterviewEngine:
    def run(self, request_text: str, existing_bundle: Optional[BundleRef]) -> InterviewTranscript: ...

class SeedWriter:
    def generate(self, transcript: InterviewTranscript, policy: PlanningPolicy) -> SeedSpec: ...

class PlanGenerator:
    def generate(self, seed: SeedSpec, policy: PlanningPolicy) -> str: # markdown
    ...

class TaskGraphBuilder:
    def generate(self, seed: SeedSpec, plan_markdown: str, policy: PlanningPolicy) -> TaskGraph: ...

class PlanningValidator:
    def validate(self, seed: SeedSpec, plan_markdown: str, graph: TaskGraph, policy: PlanningPolicy) -> PlanningValidationReport: ...
```

MVP 에서 validator 는 가능한 한 deterministic 해야 한다. LLM judge 는 FAIL/WARN 사유를 자연어로 설명하는 보조 수단으로만 사용하고, PASS/FAIL 판정 자체는 규칙 기반으로 구현한다.

6. 인터뷰 정책과 Seed Freeze 규칙

인터뷰는 ‘많이 묻는 것’이 아니라 ‘필요한 것만 명확하게 묻는 것’이 목표다. Ouroboros 의 clarity 문제 제기와 jangpm blueprint/deep-dive 의 probing 인터뷰 패턴을 결합하되, MVP 에서는 복잡한 ontology score 대신 투명한 rubric 을 채택한다.

6.1 clarity rubric

축	설명	점수
목표 명확성	무엇을 만들고 왜 필요한지가 한 문장으로 요약되는가	0~2
성공 판정 가능성	성공/실패를 testable 한 방식으로 구분할 수 있는가	0~2
범위 경계	in/out scope 가 구분되는가	0~2
제약/의존성	기술적 제약, 외부 문서/도구/사람 의존성이 드러나는가	0~2
가정/리스크	알아서 정해야 할 default 와 failure mode 가 표면화되었는가	0~2

인터뷰 종료 기본 조건은 총점 8/10 이상이면서, high severity open question 이 0 개인 경우다. 총점이 낮더라도 operator 가 명시적으로 assumption 진행을 허용하면 seed 생성으로 넘어갈 수 있다.

6.2 인터뷰 규칙

- 라운드당 질문은 최대 3 개다. 관련 질문은 묶고, 단순 반복은 금지한다.
- 이미 문서나 request 에 있는 정보는 다시 묻지 않는다.
- 사용자가 '모르겠다/알아서 해줘'라고 답하면 기본값을 제안하고 assumption 으로 기록한다.
- 질문은 generic Q&A 가 아니라 tradeoff, failure mode, dependency, acceptance criteria 를 드러내는 형태여야 한다.
- 기존 seed 또는 plan 이 발견되면 새 파일을 만들기보다 update 경로를 기본값으로 제안한다.

6.3 Seed Freeze 규칙

- seed.yaml 은 status=draft 로 생성된 뒤, validation 과 human review 를 거쳐 status=frozen 으로 승격된다.
- SEED_FROZEN 이후 plan 생성 시 seed 내용을 silent rewrite 할 수 없다.
- seed 변경이 필요하면 seed version 을 올리고 변경 사유를 run ledger 에 기록해야 한다.

7. Artifact Bundle 계약

Stage 1 산출물은 채팅 로그가 아니라 명시적인 파일 계약이어야 한다. 아래 bundle 구조를 기본값으로 사용한다.

```
artifacts/
  requests/{run_id}/request.md
  interviews/{run_id}/interview.jsonl
  seeds/{run_id}/seed.v1.yaml
  plans/{run_id}/plan.v1.md
  task-graphs/{run_id}/task-graph.v1.json
  validations/{run_id}/planning-validation.v1.json
  open-questions/{run_id}/open-questions.v1.md
  ledgers/{run_id}/run-ledger.v1.json
  manifests/{run_id}/bundle-manifest.v1.json
policies/
  planning-policy.yaml
schemas/
  seed.schema.json
  task-graph.schema.json
  planning-validation.schema.json
  run-ledger.schema.json
```

artifact	역할	상태
request.md	원본 사용자 요청과 컨텍스트	없음 / raw markdown
interview.jsonl	질문·답변 transcript	선택적 구조화 JSONL
seed.yaml	동결 가능한 초기 명세	필수 / schema validation
plan.md	인간 검토용 계획 문서	필수 / 섹션 규칙 검증
task-graph.json	dependency-aware DAG	필수 / schema + cycle 검증
planning-validation.json	gate 결과와 next actions	필수 / schema validation
open-questions.md	미해결 질문과 임시 가정	권장

run-ledger.json	상태 변화와 승인 로그	필수
bundle-manifest.json	artifact 인덱스와 버전	권장

7.1 SeedSpec 필수 필드

필드	의미
metadata.run_id/version/status	run 단위 추적과 버전 관리
problem_statement	무엇을 만들고 왜 필요한지
success_criteria[]	testable statement + measurement + verification_method
scope.in_scope/out_of_scope	범위 경계
constraints[]	시간/기술/환경 제약
assumptions[]	운영자가 수용한 기본값
dependencies[]	도구/문서/사람/서비스 의존성
risks[]	실패 모드와 대응
open_questions[]	남겨진 불확실성
deliverables[]	산출물 기대치
acceptance_test_outline[]	최소 검증 계획
approval	freeze/signoff 요구 여부

7.2 TaskGraph 필수 필드

필드	의미
id	노드 식별자. 전역 유일
title / description	태스크 정의
size	S 또는 M 만 허용
objective	이 태스크의 목적
dependencies[]	선행 노드 id
acceptance_criteria_ids[]	seed success criteria 와 traceability
outputs[]	예상 산출물
files_likely_touched[]	Stage 2 handoff 용 힌트
verify_hints[]	Stage 2 verifier 힌트
blocked_by_open_question_ids[]	해결 전 실행 불가 질문

8. Planning Gate 명세

planning gate 는 prompt convention 이 아니라 runtime rule 이어야 한다. FAIL 이면 실행을 시작하지 않고, WARN 이면 인간 승인을 기다리며, PASS 만이 기본 handoff 경로다.

ID	체크	통과 조건	실패 수준
G-01	목표 요약	problem_statement 가 존재하고 한 문장으로 핵심 목표를 설명	FAIL
G-02	success criteria testability	모든 success criteria 가 measurement 와 verification_method 를 가짐	FAIL
G-03	scope 분리	in_scope/out_of_scope 가 모두 비어 있지 않음	FAIL
G-04	assumption 명시	default decision 이 assumptions 에 기록됨	FAIL
G-05	architecture 초안	plan.md 에 modules / flow / repo layout 이 있음	FAIL
G-06	task graph 존재	최소 1 개 이상의 task node 존재	FAIL
G-07	dependency validity	모든 dependency 참조 유효, cycle 없음	FAIL
G-08	task size 제한	모든 node size $\in \{S, M\}$	FAIL
G-09	acceptance traceability	모든 task 가 acceptance_criteria_ids 를 가짐	FAIL
G-10	open question budget	high severity 0 개, medium severity ≤ 3 개	WARN/FAIL

8.1 Gate 알고리즘

```
if missing_required_seed_fields:
    FAIL
elif not all_success_criteria_are_testable:
    FAIL
elif task_graph_has_cycle:
    FAIL
elif any_task_size_not_in({"S", "M"}):
    FAIL
elif any_task_missing_acceptance_trace:
    FAIL
elif high_severity_open_questions > 0:
    FAIL
elif medium_severity_open_questions > policy.max_medium_open_questions:
    WARN
else:
    PASS
```

validator 는 PASS/WARN/FAIL 외에도 next_actions 를 함께 내보내야 한다. 예: ‘task T-04 를 두 개의 S 태스크로 분해’, ‘Q-03 을 assumption 으로 기록 후 재생성’, ‘human signoff 필요’.

9. CLI 계약

명령	설명	주요 옵션
stage1 run	요청을 받아 end-to-end bundle 생성	request-file, out-dir, policy-file
stage1 interview	인터뷰만 수행하고 transcript 저장	request-file, resume-run-id
stage1 seed	transcript → seed.yaml	transcript-file, out-file
stage1 plan	seed → plan.md + task-graph.json	seed-file, out-dir
stage1 validate	seed/plan/graph → planning-validation.json	bundle-dir, policy-file
stage1 signoff	seed freeze / plan approval 기록	run-id, actor, decision
stage1 resume	기존 bundle 재개	run-id or bundle-dir
stage1 export-schemas	schema bundle 출력	out-dir

9.1 권장 예시

```
# 신규 요청 처리
harness stage1 run --request-file ./examples/request.md --out-dir ./artifacts/demo-stage1-001 --policy-file ./policies/planning-policy.yaml

# 검증만 다시 실행
harness stage1 validate --bundle-dir ./artifacts/demo-stage1-001 --policy-file ./policies/planning-policy.yaml

# 인간 승인 기록
harness stage1 signoff --run-id demo-stage1-001 --actor "tech-lead" --decision approve-plan
```

9.2 종료 코드

exit code	의미
0	성공
2	planning gate FAIL
3	schema validation 실패
4	task graph cycle 또는 invalid dependency
5	승인 누락으로 READY 승격 불가
6	기존 bundle 충돌 / update 경로 미확정
7	예상치 못한 runtime 오류

10. 권장 기술 스택과 구현 구조

Stage 1 MVP 는 ‘빠르게 만들기 쉬운 것’보다 ‘artifact contract 가 단단한 것’을 우선해야 한다. 아래 조합이 가장 실용적이다.

기술	권장 이유
----	-------

Python 3.12+	구현 속도, schema/CLI 생태계, 테스트 친화성
Typer	타입 힌트 기반 CLI, 자동 help/validation, CLI-first MVP 에 적합
Pydantic 2	모델 정의, validation, JSON Schema 생성
JSON Schema Draft 2020-12	artifact contract 의 외부 표준
PyYAML 또는 ruamel.yaml	seed.yaml 직렬화
NetworkX 또는 자체 DAG 검사	task graph cycle validation
pytest	golden-file / validator / CLI 회귀 테스트
Rich (선택)	CLI 가독성 개선. 기능 의존은 두지 않음

10.1 저장소 구조

```

src/
  harness/
    cli.py
    policies/
      loader.py
    interview/
      engine.py
      extractor.py
      clarity.py
    planner/
      seed_writer.py
      plan_generator.py
    graph/
      builder.py
      dag.py
    validation/
      planning_gate.py
      section_rules.py
    storage/
      discovery.py
      bundle_store.py
      ledger.py
    models/
      seed.py
      task_graph.py
      validation.py
  tests/
    test_cli.py
    test_interview_policy.py
    test_seed_schema.py
    test_task_graph.py
    test_planning_gate.py
  schemas/
  policies/
  examples/

```

11. 테스트 및 acceptance suite

ID	시나리오	기대 결과
T-01	명확한 요청	인터뷰 0~1 라운드 후 PASS bundle 생성
T-02	모호한 요청	2~4 라운드 인터뷰 후 assumption/open question 반영
T-03	기존 bundle 존재	새 파일 난립 없이 update/resume 경로 제공

T-04	untestable success criteria	validator FAIL
T-05	scope 누락	validator FAIL
T-06	task graph cycle	validator FAIL
T-07	L-size task 포함	auto-split 또는 FAIL
T-08	high severity open question 남음	FAIL
T-09	medium open question 과다	WARN
T-10	schema parse 실패	repair 2 회 후 실패 코드 반환
T-11	signoff 누락	READY 승격 차단
T-12	golden-file regression	기대 bundle 구조와 diff 비교

테스트 우선순위는 validator 와 artifact contract 다. Stage 1에서는 모델 품질보다도 ‘잘못된 bundle 을 막아내는가’가 더 중요하다. 따라서 golden-file 회귀 테스트와 schema validation 테스트를 최우선에 둔다.

12. 기본 정책값 (planning-policy.yaml)

```

interview:
  max_rounds: 4
  max_questions_per_round: 3
  stop_score_threshold: 8
  allow_assumption_when_user_unsure: true

plan:
  max_tasks: 12
  allowed_task_sizes: [S, M]
  auto_split_large_tasks: false

gate:
  max_high_severity_open_questions: 0
  max_medium_severity_open_questions: 3
  require_acceptance_traceability: true
  require_scope_split: true
  require_architecture_outline: true

generation:
  schema_repair_retries: 2
  write_language: ko

approval:
  require_seed_freeze: true
  require_plan_signoff: true
  block_ready_without_signoff: true

storage:
  update_existing_bundle_by_default: true
  keep_previous_versions: true

```

threshold 는 하드코딩하지 않고 policy 파일에서 바꾸도록 설계해야 한다. 이는 SIEGE 와 NLAH 가 강조하는 policy-driven runtime 원칙과 일치한다.

13. Stage 2/3 확장을 위한 호환성 규칙

현재 필드/규칙	나중에 필요한 이유
files_likely_touched	Stage 2 worktree executor 가 충돌 가능성을 조기 판단하도록 함

verify_hints	Stage 2 verifier 에 cheap/expensive gate 힌트 전달
run-ledger.json	Stage 3 trajectory/eval 실험의 기초 trace
versioned bundle	autoresearch 계열의 keep/discard 실험에서 baseline 관리에 활용
policy 파일	budget, approval ladder, retry policy 등 Stage 2 control plane 확장에 활용

14. 2 주 MVP 구현 계획

기간	구현 범위	완료 기준
D1-D2	CLI skeleton + Pydantic models + schema export	명령 엔트리포인트와 seed/task-graph/validation schema 생성
D3-D4	Interview engine + clarity rubric	질문 정책, transcript 저장, default assumption 기록
D5-D7	Seed writer + plan generator	seed.yaml 과 plan.md 생성
D8-D9	Task graph builder + DAG validation	dependency-aware graph 생성 및 cycle 검사
D10-D11	Planning gate + validation report	10 조건 validator 와 next actions 구현
D12-D13	Bundle store + resume/update + signoff	버전 디렉터리와 ledger, 승인 기록
D14	Golden tests + docs 정리	T-01~T-12 최소 회귀 스위트

15. 부록 A - 예시 seed.yaml

```

metadata:
  run_id: demo-stage1-001
  version: 1.0.0
  status: frozen
  created_at: 2026-04-11T15:00:00+09:00
  language: ko

problem_statement: >
  사용자의 자연어 요청을 받아 실행 이전에 요구사항을 구조화하고,
  검증 가능한 planning bundle로 내보내는 Stage 1 Planning Harness MVP를 구축한다.

success_criteria:
  - id: SC-01
    statement: 필수 artifact 4종 이상을 생성한다.
    measurement: seed.yaml, plan.md, task-graph.json, planning-validation.json 존재
    verification_method: artifact_presence
    priority: must

scope:
  in_scope:
    - CLI 기반 intake/interview/seed/plan/task graph/validation 흐름
  out_of_scope:
    - 코드 수정 및 git worktree 실행

```

16. 부록 B - 예시 task-graph.json

```

{
  "run_id": "demo-stage1-001",
  "version": "1.0.0",
  "nodes": [
    {
      "id": "T-01",
      "title": "CLI skeleton 구현",
      "description": "stage1 run/interview/seed/plan/validate 명령을 등록한다.",
      "size": "S",
      "objective": "운영자가 end-to-end 명령을 호출할 수 있게 한다.",
      "dependencies": [],
      "acceptance_criteria_ids": [
        "SC-01"
      ],
      "outputs": [
        "src/harness/cli.py",
        "tests/test_cli.py"
      ],
      "files_likely_touched": [
        "src/harness/cli.py",
        "tests/test_cli.py"
      ],
      "verify_hints": [
        "pytest tests/test_cli.py"
      ],
      "blocked_by_open_question_ids": [],
      "owner_role": "planner"
    },
    {
      "id": "T-02",
      "title": "Seed schema 및 serializer 구현",
      "description": "Pydantic 모델과 YAML/JSON serializer를 추가한다.",
      "size": "S",
      "objective": "schema-valid seed artifact를 생성한다.",
      "dependencies": [
        "T-01"
      ],
      "acceptance_criteria_ids": [
        "SC-01",
        "SC-04"
      ],
      "outputs": [
        "src/harness/models/seed.py",
        "schemas/seed.schema.json"
      ],
      "files_likely_touched": [
        "src/harness/models/seed.py",
        "schemas/seed.schema.json"
      ],
      "verify_hints": [
        "pytest tests/test_seed_schema.py"
      ],
      "blocked_by_open_question_ids": [],
      "owner_role": "planner"
    },
    {
      "id": "T-03",
      "title": "Interview engine 구현",
      "description": "clarity rubric, round budget, question policy를 구현한다.",
      "size": "M",
      "objective": "명확하지 않은 요청을 구조화된 transcript로 변환한다.",
      "dependencies": [
        "T-01"
      ],
      "acceptance_criteria_ids": [
        "SC-01"
      ],
      "outputs": [
        "src/harness/interview/engine.py",
        "tests/test_interview_policy.py"
      ],
      "files_likely_touched": [
        "src/harness/interview/engine.py",
        "tests/test_interview_policy.py"
      ],
      "verify_hints": [
        "pytest tests/test_interview_policy.py"
      ],
      "blocked_by_open_question_ids": [],
      "owner_role": "planner"
    }
  ]
}

```

17. 부록 C - 예시 planning-validation.json

```

{

```

```

"run_id": "demo-stage1-001",
"overall_status": "PASS",
"checks": [
  {
    "id": "G-01",
    "name": "목표 요약 존재",
    "status": "pass",
    "detail": "problem_statement 가 존재한다.",
    "evidence": [
      "seed.problem_statement"
    ]
  },
  {
    "id": "G-02",
    "name": "testable success criteria",
    "status": "pass",
    "detail": "모든 success criteria가 measurement를 가진다.",
    "evidence": [
      "seed.success_criteria[*].measurement"
    ]
  },
  {
    "id": "G-03",
    "name": "scope 분리",
    "status": "pass",
    "detail": "in_scope/out_of_scope가 모두 채워져 있다.",
    "evidence": []
  },
  {
    "id": "G-04",
    "name": "assumptions 명시",
    "status": "pass",
    "detail": "2개의 assumption이 기록되었다.",
    "evidence": []
  },
  {
    "id": "G-05",
    "name": "architecture 초안",
    "status": "pass",
    "detail": "plan.md에 modules, flow, repo layout이 포함되어 있다.",
    "evidence": []
  },
  {
    "id": "G-06",
    "name": "task graph 존재",
    "status": "pass",
    "detail": "6개 노드 DAG가 생성되었다.",
    "evidence": []
  },
  {
    "id": "G-07",
    "name": "dependency validity",
    "status": "pass",
    "detail": "cycle이 없고 모든 참조가 유효하다.",
    "evidence": []
  },
  {
    "id": "G-08",
    "name": "oversized task",
    "status": "pass",
    "detail": "모든 태스크가 S 또는 M이다.",
    "evidence": []
  },
  {
    "id": "G-09",
    "name": "eval/test outline",
    "status": "pass",
    "detail": "acceptance test outline이 존재한다.",
    "evidence": []
  },
  {
    "id": "G-10",
    "name": "open question budget",
    "status": "warn",
    "detail": "중간 심각도 open question 1건이 남아 있으나 assumption으로 처리 가능하다.",
    "evidence": []
  }
]
"failed_checks": [],
"warnings": [
  "Q-01은 Stage1 로컬 파일만 지원한다는 가정으로 진행한다."
],
"next_actions": [
  "인간 승인 후 bundle 상태를 READY로 승격한다.",
  "T-04와 T-05를 먼저 구현한다."
],
"generated_at": "2026-04-11T15:10:00+09:00"
}

```

18. 참고 소스

- [R0] 기존 3 단계 검토 보고서 (사용자 제공 PDF)
- [R1] SlopCodeBench - <https://arxiv.org/abs/2603.24755>
- [R2] Paperclip - <https://github.com/paperclipai/paperclip>
- [R3] jangpm-meta-skills README - <https://github.com/byungjunjang/jangpm-meta-skills>
- [R4] jangpm blueprint SKILL - <https://raw.githubusercontent.com/byungjunjang/jangpm-meta-skills/main/.claude/skills/blueprint/SKILL.md>
- [R5] jangpm deep-dive SKILL - <https://raw.githubusercontent.com/byungjunjang/jangpm-meta-skills/main/.claude/skills/deep-dive/SKILL.md>
- [R6] Agent Skills - <https://github.com/addyosmani/agent-skills>
- [R7] Ouroboros - <https://github.com/Q00/ouroboros>
- [R8] Superpowers - <https://github.com/obra/superpowers>
- [R9] SIEGE - <https://github.com/nutshells3/Swarm-Intelligence-Engine-with-Gated-Execution>
- [R10] Natural-Language Agent Harnesses - <https://arxiv.org/abs/2603.25723>
- [R11] OpenAI Structured Outputs - <https://developers.openai.com/api/docs/guides/structured-outputs>
- [R12] OpenAI Working with Evals - <https://developers.openai.com/api/docs/guides/evals>
- [R13] OpenAI Evaluation Best Practices - <https://developers.openai.com/api/docs/guides/evaluation-best-practices>
- [R14] LangSmith Trajectory / Evaluation Approaches - <https://docs.langchain.com/langsmith/trajectory-evals> ; <https://docs.langchain.com/langsmith/evaluation-approaches>
- [R15] Pydantic JSON Schema - https://pydantic.dev/docs/validation/latest/concepts/json_schema/
- [R16] Typer - <https://typer.tiangolo.com/>
- [R17] JSON Schema Draft 2020-12 - <https://json-schema.org/draft/2020-12>